JEMCO Complaint & NCR System — Technology Plan

Oct 10, 2026 · @Adpedia

Decision summary

JEMCO builds a custom complaint and NCR system as two separately deployed applications, a Next.js frontend and a Node.js API, on one PostgreSQL database, to go live in about 18 weeks. The build implements every Must requirement in the companion document *JEMCO Complaint & NCR Management — Requirements*.

Item	Decision
Approach	Custom build; the ERPNext spike in Phase 0 is a 2-day check that confirms it, not a competing option
Frontend	Next.js 15 (App Router), TypeScript, deployed on its own
Backend	Node.js 22 with NestJS and Prisma, a REST API, deployed on its own
Database	PostgreSQL 16, one database, with triggers for the audit log and the state machine
Attachments	S3-compatible object storage, uploaded straight from the browser or phone
Users	About 30 to 50 staff: Sales, Production, Powder Coating, Warehouse, Purchasing, Logistics, QA, management
First release	Intake, triage, action, verification, SLA clocks, notifications, management dashboard, PDF of F/QP/004-01
Timeline	Phase 0 starts on sign-off; go-live at week 18, pilot department live at week 12

User-friendly is a requirement here, not a preference: most users are production, warehouse and sales staff who fill in one paper form a month and have never used a ticketing tool. The system therefore asks one question at a time, uses JEMCO's own

words (NCR, PIC, Lot Disposition), shows each person only what they must do next, and never needs a manual. The rules that achieve this are in the UX principles section, and every screen is checked against them before it ships.

Who uses it

Five kinds of people use the system, and only one of them (QA) uses it every day. The design is driven by the other four, who open it rarely and must succeed first time.

User	Who they are	How often	Device	Comfort with software	What the screen must do for them
Issuer	Sales staff logging a customer complaint; any staff member reporting an internal problem	A few times a month	Office PC; phone for Sales on site	Uses email and WhatsApp; may not have used a web form	One guided form, one question per step, photo upload from the phone, a clear "sent, here is your number" at the end
PIC (person in charge)	Supervisor in Production, Powder Coating, Warehouse, Purchasing or Logistics	When assigned, 2 to 6 times a month	Phone on the floor, shared PC in the office	Low; may not type fluently in English	A to-do list with the oldest or most urgent item first, big buttons, plain words, Five-Why prompts written as questions, voice-to-text friendly fields
Department manager	Head of each department	Weekly	PC, phone	Medium	Their department's open items and who is late, nothing else by default
QA Coordinator and QA manager	The daily operators: validate, assign, verify, close, configure	All day	PC with two screens	High; today they run the paper process	Queues, keyboard-fast forms, bulk views, the admin screens, the full record history
Management and auditor	Managing Director; internal and external ISO auditors	Monthly; audit days	PC, tablet	Medium	The dashboard, the register, a PDF of any NCR in the form layout, read-only everywhere

Three facts shape every design choice: a PIC's first contact with the system is a notification on their phone, so the link in it must open the exact item with nothing to find; the issuer and PIC never need to understand the whole workflow, only their next step; and QA is the only user who should see a table with filters.

User experience principles

Twelve rules apply to every screen; a screen that breaks one is not finished. They are written so a non-technical reviewer at JEMCO can check them.

#	Rule	What it looks like in the app
1	One job per screen	Raising a complaint is a stepper: what happened, where, who, how bad, photos, done. No tabs, no side panels, no "advanced"
2	Next step first	Every user lands on "Your to-do" with the most urgent item at the top and one button on each: Start, Continue, Submit. The full list is one tap away, never the landing page
3	JEMCO's words, no jargon	Labels say NCR, PIC, Lot Disposition, Respond by, as the paper form does. No "ticket", "workflow", "entity", "status code"
4	Plain-language choices with help on the spot	Priority shows the four levels with the form's own one-line meaning under each; Source and Process are big tappable cards, not a dropdown of nine
5	Ask only what is needed now	Supplier name appears only when Source is Supplier; CEF number only when Process is Sales. Required fields are few and marked; everything else is optional and collapsed
6	Photos in three taps	Camera button on the form, photo attached, thumbnail shown. Drawings and PDFs are drag-and-drop on PC
7	Nothing is ever lost	Drafts save as you type; closing the phone and returning resumes where you were. A saved banner confirms it

#	Rule	What it looks like in the app
8	Confirmation you can keep	Submitting shows the number (CMP-2026-0042), what happens next, and when to expect a reply, with a button to copy or WhatsApp it
9	Status in colour and words	Each item carries one chip: green on track, amber due soon, red overdue, grey closed, always with the words beside the colour
10	Big targets, readable type	48 px tap targets, 16 px minimum body text, high contrast; works one-handed on a phone and with the floor's gloves off
11	No dead ends	Every error says what to do: "Add at least one photo or write 50 characters" not "Validation failed". Every empty list says why it is empty
12	Two languages from day one	English and Bahasa Malaysia switchable per user; all labels come from a translation file, never typed into components

How the rules are enforced: a clickable prototype of the issuer and PIC journeys is tested with three real JEMCO staff in Phase 0 before any code; each sprint review walks one real user through the new screens; and a checklist of the twelve rules is part of the definition of done for every screen. **Decide:** confirm Bahasa Malaysia as the second language and whether Mandarin is needed.

Brand and visual direction

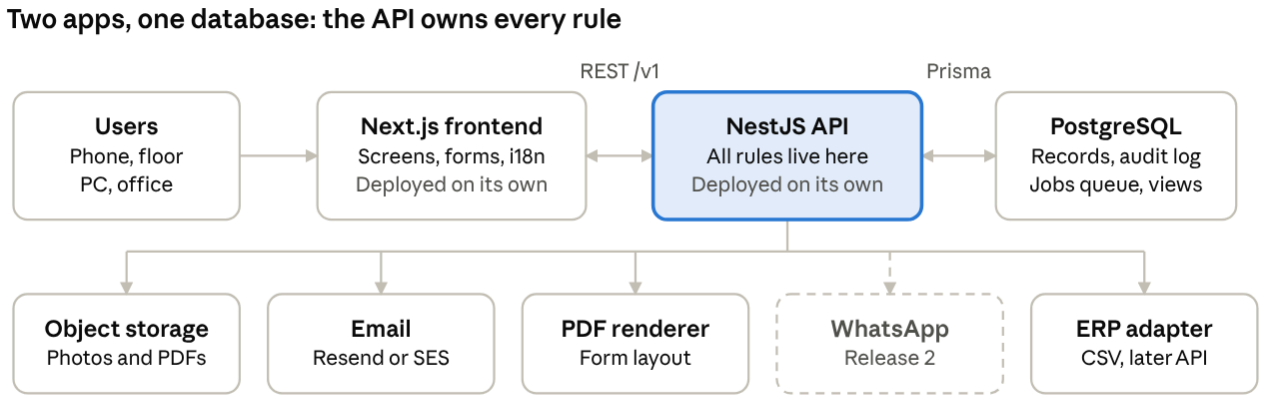
The JEMCO logo is a red italic wordmark set in a charcoal gear with a red hub, under the tagline "Technology Solutions". Measured from the supplied file, the brand red averages #CD1918 (shaded from #A01010 to #E01010) and covers 74% of the mark; the charcoal averages #474749; highlights are #F0F0F0. The app uses the red sparingly, as the one accent, so that it still means "act here" and never "error".



Token	Value	Used for
Brand red	#CD1918	Primary buttons, the active step, the logo; never for text on white smaller than 18 px
Red dark	#A01010	Button hover and pressed states
Charcoal	#474749	Headings, the top bar on PC
Ink	#1F1F23	Body text
Grey 60	#6B6B70	Secondary text, help lines
Grey 20	#E6E6E9	Borders, dividers
Surface	#F7F7F8	Page background; cards are white on it
Status green	#1E8E3E	On track
Status amber	#C77700	Due soon (50 to 80% of the clock)
Status red	#B3261E	Overdue or breached; deliberately a different red from the brand red
Status grey	#8A8A8F	Closed or rejected

Type: Inter for the interface (free, clear at small sizes, has Bahasa Malaysia diacritics), 16 px body, 20 px form questions, 28 px page titles; the logo's italic style is not used in the UI. Tone of voice: short sentences in the second person ("Tell us what happened"), no exclamation marks, numbers as digits. Dark mode is not in scope; the floor and office are bright environments and one theme keeps contrast checks simple. **Decide:** whether JEMCO has a brand guideline that fixes the exact red and a corporate typeface; the values above are measured from the logo and will be replaced by official ones if they exist.

Architecture



architecture · 2 apps, 1 database, 5 services

The browser talks only to the Next.js frontend; the frontend talks only to the API; the API is the only thing that touches the database and the services under it. WhatsApp is drawn dashed because it is wired in the notification module but not switched on in release 1.

Frontend: Next.js

The frontend is a standalone Next.js application that talks only to the API; it holds no business rules, so a change to an SLA or a routing rule never needs a frontend release.

Concern	Choice	Reason
Framework	Next.js 15, App Router, TypeScript, React 19	Server components for fast first load on the floor's phones; file-based routes map one-to-one to the screens below
Styling	Tailwind CSS with the brand tokens as the theme; shadcn/ui as the base component set	Accessible components out of the box (focus, keyboard, screen reader); tokens enforce the palette
Forms	React Hook Form + Zod schemas shared with the API package	One definition of "what is valid" on both sides; errors in plain language from one message file
Data	TanStack Query against the REST API; optimistic updates on the to-do list	Lists feel instant; retries and offline-tolerant drafts handled in one place
Auth	Session cookie issued by the API; middleware redirects by role	Frontend never holds a secret; the API decides what each role sees

Concern	Choice	Reason
i18n	next-intl with `en` and `ms` message files	Rule 12: no label is typed into a component
Charts	Recharts on data already aggregated by the API	Dashboard renders in one request per tile
PWA	Installable on the phone home screen; drafts cached	A PIC opens it like an app, and a lost signal does not lose a half-written root cause

Screens

Route	Who	What it does
`/todo`	Everyone, landing page	Your next actions, most urgent first, one button each
`/complaints/new`	Issuer	Six-step stepper: What happened, Where (Process cards), Who (customer or supplier), How bad (Priority cards), Photos, Review and send
`/complaints/[no]`	Issuer, QA	The complaint, its status in words, and the NCR link once validated
`/qa/triage`	QA	Queue of Submitted; validate, reject with reason, set priority, assign; keyboard-first
`/ncr/[no]`	PIC, QA, managers	The NCR as one scrolling page in the paper form's order: issue, action, verification; each part editable only by its owner in its state
`/ncr/[no]/action`	PIC	Guided action form: containment, Five-Why as five questions, actions as a checklist, evidence photos, submit
`/qa/verify`	QA	Queue of Awaiting verification; disposition, result, justification, close or return
`/dept`	Department manager	Department's open items, who is late, workload per PIC
`/dashboard`	QA, management, auditor	Tiles and charts from the Requirements document, each clickable to the records

Route	Who	What it does
`/register`	QA, auditor	Every NCR, filter by any field, export Excel or PDF
`/admin/*`	QA manager	Dropdown lists, routing map, SLA table, calendar, users and roles

Components that keep it simple

- **Stepper**: one question per step, progress shown as "Step 2 of 6", back always available, draft saved on every step.
- **ChoiceCards**: large tappable cards with a title and one line of help for Process, Source, Priority and Disposition; replaces every dropdown a layman would see.
- **StatusChip**: colour plus words, used everywhere a state appears, driven by one lookup.
- **ClockBar**: the SLA as a bar with the due date in words ("Due Thursday, 3 days left"), never a raw timestamp.
- **PhotoField**: camera first on phones, drag-and-drop on PC, thumbnails with delete, compression before upload.
- **RecordTimeline**: the change history as a readable story ("Aishah assigned this to Rahman on 12 Oct"), from the audit log.

Backend: Node.js API

The API is a separate NestJS service that owns every rule: who may do what, which state follows which, when a clock breaches, who gets told. The frontend and any future client (WhatsApp bot, ERP sync, mobile app) use the same endpoints.

Concern	Choice	Reason
Runtime and framework	Node.js 22 LTS, NestJS 10, TypeScript	Modules, dependency injection and guards give the structure a handover needs; large hiring pool in Malaysia
API style	REST with OpenAPI generated from the code; versioned under `/v1`	Simple to consume, easy to test with any tool, contract visible to the frontend team
ORM and migrations	Prisma with versioned SQL migrations committed to the repo	Typed queries; every schema change is a reviewed file the auditor can see

Concern	Choice	Reason
Validation	Zod schemas in a shared package used by both API and frontend	One definition of valid input
Auth	Email and password with Argon2 now; Google or Microsoft SSO added when JEMCO confirms its identity provider; session cookie, httpOnly, 12-hour life, refreshed on activity	No tokens in the browser; SSO is a module, not a rewrite
Authorisation	Role guard per endpoint plus a record-scope check (own department, own items)	Implements the permissions matrix in the Requirements document exactly
Jobs	pg-boss, a Postgres-backed queue in the same database; a scheduler module runs clock checks every 15 minutes and digests on schedule	No extra infrastructure to run or back up
Email	Nodemailer behind an adapter; Resend or SES as the provider	Provider swap is one file
PDF	Playwright renders an HTML template of F/QP/004-01 to PDF, stored in object storage and cached per record version	Pixel-faithful form; regenerated only when the record changes
Logging and errors	Pino structured logs; Sentry for exceptions; request id on every log line and API response	A support call can be traced from the user's screen to the log

Modules

Module	Owns
`auth`	Login, sessions, password reset, SSO
`users`	Users, roles, departments, out-of-office flag
`complaints`	Part 1 intake, drafts, numbering, rejection
`ncr`	The NCR record, Parts 2 and 3, linking, reopen

Module	Owns
`workflow`	The state machine: allowed transitions per role, guards, side effects
`actions`	Action items under an NCR
`sla`	Clock creation, pause, extension, breach detection, working-day calendar
`routing`	Process-to-department map, PIC and backup, absence fallback, escalation ladder
`notifications`	Event to recipient to channel; templates in `en` and `ms`; delivery log
`attachments`	Presigned upload URLs, virus scan hook, retention
`reports`	Dashboard tiles, register export, supplier scorecard, management pack
`pdf`	Form-layout rendering
`admin`	Lookup lists, SLA table, calendar, routing map, feature flags
`audit`	Read access to the change log; nothing writes to it except the database trigger

The state machine

All transitions live in one table, `state_transitions` (from state, to state, role allowed, reason required). The `workflow` module checks the table, applies the transition in one database transaction with its side effects (start or stop a clock, queue a notification, write the transition record), and rejects anything else. Adding a state or changing who may move a record is a data change with a migration, not new code paths across the app.

Database: PostgreSQL

One PostgreSQL 16 database holds records, configuration, the audit log and the job queue; the audit log and the state machine are enforced by the database itself so that no application bug or direct SQL edit can bypass them.

Tables

Group	Tables	Notes
Records	`complaints`, `ncrs`, `actions`, `attachments`, `ncr_links`, `effectiveness_reviews`	The three record types from the Requirements document plus their children
Workflow	`states`, `state_transitions`, `transition_log`	Transitions as data; every move logged with who, when, reason
Time	`sla_policies`, `sla_clocks`, `working_calendar`, `clock_extensions`	One `sla_clocks` row per clock per record: started, due, paused, breached; materialised so alerts and reports agree
People	`users`, `roles`, `user_roles`, `departments`, `routing_map`, `absences`	Primary and backup PIC per department
Configuration	`lookup_lists`, `lookup_options` (type, source, process, disposition, root cause, units), `settings`	Edited in admin screens, never in code
Communication	`notification_events`, `notification_deliveries`, `templates`	Who was told what, when, by which channel, and whether it was delivered
Audit	`change_log`	Append-only: table, row, field, old, new, user, time; written by trigger
Queue	`pgboss.*`	pg-boss schema, same database

Rules enforced in the database

- **Audit trigger** on every record table writes to `change_log`; the log table has no UPDATE or DELETE grant for any application role.
- **No deletes** on record tables for the application role; rows are closed or rejected.
- **Transition check**: a trigger on `ncrs.state` rejects any change not present in `state_transitions`, even if the API is bypassed.
- **Numbering** by sequence per year inside a transaction, so two simultaneous submissions never share a number.

- **Row-level security** as a second line behind the API's role guard: a department-scoped role can only read its department's rows.
- **Working-day function** `working_days_between(start, end)` and `add_working_days(start, n)` in SQL, used by both the clock jobs and the reports, so there is one definition of a working day.

Reporting views

Dashboard tiles read from materialised views refreshed every 15 minutes (`v_open_by_state`, `v_overdue`, `v_cycle_time`, `v_sla_attainment`, `v_by_process`, `v_by_source`, `v_root_cause`, `v_cost`, `v_recurrence`, `v_return_rate`). The register and exports read live tables. Keeping aggregation in SQL views means a KPI's definition is one reviewable statement, and the same number appears on the screen, in the Excel export and in the management pack.

Backups and retention

Daily full backup with point-in-time recovery, retained 35 days; a monthly backup retained for the record retention period JEMCO sets (Requirements, decision 11). A restore is rehearsed before go-live and once a year after.

Integrations

Every external service sits behind an adapter interface in the API, with a working default and a swap path, so that a provider change or a JEMCO decision never reaches the frontend.

Integration	Default in release 1	Swap path	Lead time to start now
File storage	S3-compatible bucket (Cloudflare R2 or the host's object storage); presigned upload from browser or phone; images compressed client-side to under 2 MB	Any S3 API, or a local volume if on-prem is chosen	None
Email	Resend (or AWS SES) from `ncr@jemco` domain with SPF, DKIM and DMARC set	Any SMTP provider	2 to 3 days for DNS records on JEMCO's domain

Integration	Default in release 1	Swap path	Lead time to start now
PDF	Playwright in the API container rendering the F/QP/004-01 template	Managed PDF service	None
WhatsApp	Not in release 1; the notification module already writes a `whatsapp` channel row that is skipped	Meta WhatsApp Business Cloud API via a provider such as Twilio; template messages must be pre-approved	2 to 6 weeks for Meta business verification; apply in Phase 0
ERP lookup	CSV import of customers, suppliers and open PO and DO numbers, uploaded weekly by Purchasing in the admin screen	Live REST adapter or nightly database sync once JEMCO names the ERP and its API	Depends on the ERP; **Decide**
Identity	Email and password; a company email domain allow-list	Google Workspace or Microsoft 365 SSO via OpenID Connect	1 day once JEMCO confirms the provider
Calendar	Malaysian public holidays loaded for the current and next year, editable by QA	Automatic feed	None
Error and uptime monitoring	Sentry for exceptions; an uptime check on `/health` every minute	Any	None

The ERP question is the one that most changes scope: with a live API the PO number is validated as it is typed and the customer is filled in; with CSV it is validated against last week's list and QA corrects mismatches at triage. Both are acceptable for release 1; the adapter interface is the same.

Hosting, environments and DevOps

The system runs as two containers and a managed database, in a region JEMCO is comfortable with; the recommended host is a Singapore or Malaysia region of a managed platform, with a single-VPS option if on-prem or local residency is required.

Option	What runs where	Monthly run cost (indicative)	Choose when
A. Managed platform (recommended)	Frontend on Vercel; API container on Railway or Render (Singapore); PostgreSQL on Neon or Supabase (Singapore); R2 for files	About USD 80 to 150	JEMCO accepts cloud hosting in Singapore; nobody at JEMCO maintains servers
B. Single VPS in Malaysia	One VM (4 vCPU, 8 GB) running Docker Compose: frontend, API, PostgreSQL, MinIO, Caddy for TLS; managed with Coolify	About USD 40 to 80 plus a backup bucket	Data must stay in Malaysia, or on-prem is required
C. On-prem server	Same Compose stack on a JEMCO server	Hardware plus IT time	JEMCO has IT staff and insists on on-prem

Whichever is chosen, the application code is identical; only the Compose file and environment variables differ.

Environments

Environment	Purpose	Data
Local	Developer machines; Docker Compose for PostgreSQL and MinIO	Seed data
Staging	Every merge to `main`; JEMCO reviews and tests here	Anonymised copy of production once live; seed data before
Production	Tagged releases only	Real

Pipeline

One GitHub repository with two applications and a shared package (`apps/web`, `apps/api`, `packages/shared`), managed with pnpm workspaces and Turborepo. On every pull request: lint, type check, unit tests, API contract tests against a throwaway database, and a Playwright run of the issuer and PIC journeys on a phone viewport. On merge to

`main`: build both containers, run migrations against staging, deploy. On a release tag: the same against production, with a manual approval step and an automatic rollback if `/health` fails within 5 minutes.

Operations

- Logs and errors in one place (Sentry plus the host's log viewer) with the request id shown on any error screen so a user can quote it.
- Uptime check every minute; an alert to Adpedia and the QA manager after 3 failures.
- Dependency updates monthly; a security patch within 7 days of disclosure.
- A one-page runbook: how to restore a backup, rotate a secret, add a user, and what to do if email stops.

Decide: hosting option A, B or C, and the region.

Security and compliance controls

The controls below map each obligation in the Requirements document's compliance section to the mechanism in this stack that satisfies it, so the auditor can be shown the mechanism rather than told about it.

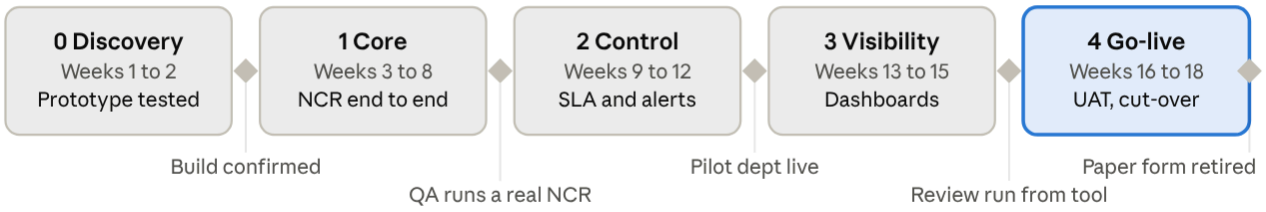
Obligation	Mechanism	Where
Records protected from unauthorised change	Role guard per endpoint; row-level security; closed records read-only at the database	API, PostgreSQL
Changes traceable	Trigger-written `change_log`, no update or delete grant; RecordTimeline shows it to QA and auditors	PostgreSQL, frontend
Signature equivalence	Personal login, no shared accounts enforced by the domain allow-list and one session per user; the transition log records user, time and IP	API
Retention	Record tables never deleted; attachments kept for the retention period; monthly backups kept the same period	PostgreSQL, storage

Obligation	Mechanism	Where
Access control	Argon2 password hashing, 12-character minimum, lockout after 10 failures; SSO when confirmed; sessions httpOnly, SameSite, 12-hour life	API
Data in transit and at rest	TLS everywhere; database and storage encrypted at rest by the host; secrets in the host's secret store, never in the repository	Hosting
Input safety	Zod validation on every endpoint; parameterised queries only through Prisma; file type and size checked on upload; images re-encoded server-side	API
Least privilege	The API connects with a role that cannot drop tables or touch the audit log; migrations run with a separate role in the pipeline	PostgreSQL
Personal data	Customer and supplier contacts limited to name, company and one contact field; auditor export masks them; data processing noted for PDPA	API, reports
Availability	Daily backups with point-in-time recovery; restore rehearsed; uptime alerting	Hosting
Change control	Every change is a reviewed pull request; releases are tagged; migrations are files in the repository	GitHub

A short penetration test by an independent tester is run on staging in Phase 4, scoped to authentication, authorisation and file upload; findings rated high or above are fixed before go-live.

Delivery plan

Pilot department live at week 12, go-live at week 18



roadmap · 5 phases, 5 gates, 18 weeks

Each gate is a demonstration, not a document: Phase 1 ends when QA runs a real NCR through staging; Phase 2 ends when one department has used the tool for two weeks; Phase 3 ends when a management review is run from the dashboard. Sprints are two weeks, each ending with one real user walking through the new screens.

Phase	Weeks	Built	Gate
0 Discovery and design	1 to 2	Constraint sheet, ERPNext spike, clickable prototype tested with 3 staff, stack decision record, repository, pipeline, staging	Build confirmed; decisions T1 to T3 made
1 Core	3 to 8	Auth and roles; intake stepper; QA triage; PIC action form; QA verification; state machine; audit log; attachments; record page and register	QA runs a real NCR end to end on staging
2 Control	9 to 12	SLA clocks and alerts; routing map and backups; escalation; email; PDF of the form; admin screens	One department live for 2 weeks on production
3 Visibility	13 to 15	Management, department and QA dashboards; Excel and PDF exports; supplier scorecard; management review pack	A management review run from the tool

Phase	Weeks	Built	Gate
4 Go-live	16 to 18	Effectiveness review timer; UAT by every department; open paper NCRs entered; training; penetration test; QP/004 revised; cut-over	Paper form retired; outage contingency documented
Release 2	after go-live	WhatsApp alerts; live ERP adapter; SSO if not already; customer acknowledgement portal	Business case per item

Team, estimate and risks

One full-stack developer, a part-time designer and a product owner from JEMCO's QA team deliver the first release in about 18 weeks; a second developer shortens Phases 1 to 3 by roughly 6 weeks but JEMCO's decisions, not code, set the floor.

Role	Who	Load	Responsibility
Product owner	JEMCO QA Coordinator or QA manager	1 day a week, 2 in Phases 0 and 4	Decisions, acceptance, user testing, configuration
Lead developer	Adpedia	Full time	API, database, pipeline, half the frontend
Frontend developer (optional second)	Adpedia	Full time, Phases 1 to 3	Screens and components
UX designer	Adpedia	2 days a week, Phases 0 to 2	Prototype, user tests, design tokens, screen reviews against the 12 rules
Tester	Adpedia or JEMCO	2 days a week, Phases 2 to 4	Test cases from the Requirements Must list, UAT support

Effort by phase

Phase	Weeks	Developer weeks	Main deliverables
0 Discovery and design	2	2	Constraints, ERPNext spike, prototype tested with 3 staff, stack record, repository and pipeline
1 Core	6	6 to 10	Auth, roles, intake stepper, triage, action form, verification, state machine, audit, attachments, register
2 Control	4	4 to 7	SLA clocks, alerts, routing, escalation, email, PDF, admin screens; pilot department live
3 Visibility	3	3 to 5	Dashboards, exports, supplier scorecard, management pack
4 Go-live	3	3	Effectiveness review, UAT, migration of open paper NCRs, training, penetration test, cut-over
Total	18	18 to 27	

Risks

Risk	Likelihood	Effect	Mitigation
JEMCO decisions stall	High	Every later phase slips	Phase 0 workshops have fixed dates; a default applies to any decision not made by its date and is recorded as such
Users reject the tool as harder than paper	Medium	Paper continues in parallel; data incomplete	Prototype tested before code; pilot with one department; the 12 rules in every review
ERP has no usable API	Medium	Manual lookups stay manual	CSV adapter in release 1; live adapter later

Risk	Likelihood	Effect	Mitigation
WhatsApp verification delayed	High	Urgent alerts by email only at go-live	Apply in Phase 0; email and in-app are the Must channels
ISO audit falls mid-build	Medium	Auditor sees two systems	Keep the paper form live until cut-over; show the tool as a documented transition
Single developer unavailable	Medium	Schedule slips	Repository, runbook and pipeline documented from week 1; second developer can join at any phase boundary
Scope grows during build	High	Release 1 slips	Should items wait for release 2; new Musts need a change note the product owner signs

Open decisions and next steps

Eight technology decisions are JEMCO's before the first sprint; the first three are needed in Phase 0 week 1, the rest by the end of Phase 0. The business decisions from the Requirements document (data model, SLA table, routing map, dashboard numbers) remain open alongside these.

#	Decision	Needed by	Suggested owner	Status
T1	Hosting option A, B or C, and region	Phase 0, week 1	Managing Director, JEMCO IT	Open
T2	Identity provider: Google Workspace, Microsoft 365, or email and password	Phase 0, week 1	JEMCO IT	Open
T3	Which ERP holds POs, DOs, customers and suppliers, and whether it has an API	Phase 0, week 1	Purchasing, JEMCO IT	Open
T4	Second language: Bahasa Malaysia confirmed; Mandarin needed or not	Phase 0	QA manager	Open

#	Decision	Needed by	Suggested owner	Status
T5	Official brand colours and typeface, if a guideline exists	Phase 0	Management	Open
T6	WhatsApp: apply for Meta business verification now, or defer to release 2	Phase 0	Management	Open
T7	Who maintains the system after go-live: Adpedia retainer, JEMCO IT, or both	Phase 0	Management	Open
T8	Second developer: yes for an 12-week delivery, no for 18 weeks	Phase 0	Management, Adpedia	Open

Next steps:

- [ ] JEMCO signs off this plan and the Requirements document by Oct 24, 2026
- [ ] Phase 0 kick-off workshop: constraints T1 to T3, business decisions 1 to 4, names of the three staff who will test the prototype
- [ ] Adpedia creates the repository, pipeline and staging environment in week 1 of Phase 0
- [ ] Designer builds the issuer and PIC prototype; tested with the three staff in week 2
- [ ] ERPNext spike written up in 2 days and shown at the end of Phase 0 as confirmation of the build decision
- [ ] Sprint 1 of Phase 1 starts the Monday after Phase 0 sign-off